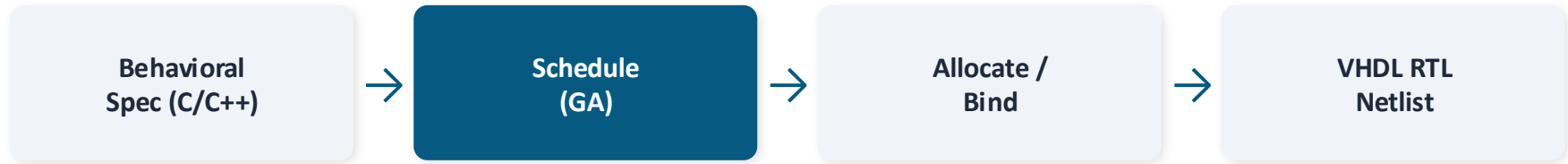


High-Level Synthesis with Genetic Algorithm Scheduling

Moshiour Rahman Prince

What is High-Level Synthesis?



Scheduling

When does each operation run? (NP-hard)

Allocation

How many functional units are available?

Binding

Which unit executes which operation?

The Scheduling Problem



Input: a task graph (DAG) — nodes are operations, edges are dependencies

Constraint 1

Precedence

A task cannot start before
its predecessors finish

Constraint 2

Resource Bound

At most r_k units of type k
may be active at once

ASAP = earliest valid start | **ALAP** = latest valid start | **Mobility** = range between them

Finding the minimum-latency schedule under resource constraints is NP-hard (Hwang et al., 1991)

Genetic Algorithm: The Idea

1

Selection

Pick the fittest candidates

2

Crossover

Mix genes of two parents

3

Mutation

Randomly tweak a gene

4

Elitism

Keep the best 10% unchanged

Fitness Function

$$\text{Fitness} = 1 / (1 + C + \lambda \cdot V)$$

C = weighted cost (latency + waiting + signal changes)

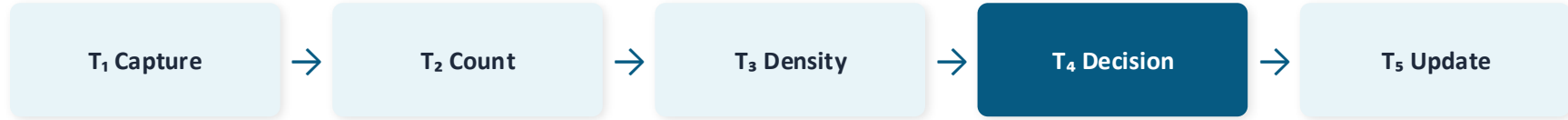
V = constraint violations (precedence + resources)

λ = 1000 (large penalty weight)

Chromosome = vector of start cycles (one gene per task)

Smart Traffic Signal Controller

Per-lane task pipeline (5 tasks, rigid chain):



T₄ Decision is the bottleneck: only 2 units for 4 lanes

Lane	Vehicles	Status	Green
A	20	Congested	30 s
B	12	Nominal	20 s
C	30	Congested	30 s
D	5	Nominal	20 s

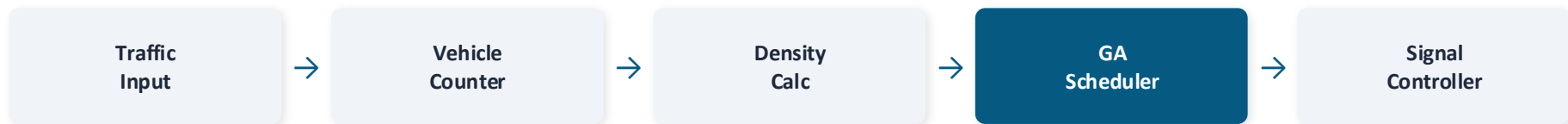
Key Insight

One lane = rigid chain, NO scheduling freedom.

The problem is non-trivial ONLY because 4 lanes share 2 decision units.

The GA must decide: which 2 lanes go first?

C++ Implementation



Fitness Evaluation (Listing 2)

```
double evaluate(Schedule& c, TaskGraph& g, ...)  
    int T = 0, viol = 0;  
    for (int i = 0; i < N; ++i) {  
        T = max(T, s + d);          // makespan  
        active[{t, type}]++;        // resource usage  
        if (pred violated) ++viol;  
    }  
    for (auto& kv : active)          // resource check  
        if (kv > units[type]) viol += overflow;  
    cost = wL*T + wW*wait + wS*changes;  
    fitness = 1.0 / (1 + cost + lambda*viol);
```

GA Parameters: P = 200 G = 400 mutation = 0.2 λ = 1000

Results

Scheduling-Level: GA vs FIFO

Metric	FIFO	GA
Latency (steps)	5	5
Makespan (cycles)	6	6
Waiting time	236	218
Signal changes	3	1
Violations	0	0

-7.6%

waiting time reduction

Traffic-Level: Policy Comparison

Metric	Fixed	FIFO	Adaptive
Waiting (s)	48	41	29
Throughput	38	44	57

From Schedule to Hardware

The GA does NOT run in hardware. It runs offline in C++. Its OUTPUT becomes the FSM.

1 Schedule → FSM

GA output becomes a state machine with states like DECIDE_A_C, DECIDE_B_D, UPDATE_ABD

2 Function → FU

greenTime() becomes a VHDL comparator + mux (density_calc)
2 instances shared by 4 lanes

3 Routing → Mux

FSM drives lane-select signals to steer input multiplexers
= the BINDING step of HLS

This IS what HLS does — we just did it manually to show the concept.



Lane A = 30s, B = 20s, C = 30s, D = 20s — ALL TESTS PASSED



Conclusion

- ✓ Behavioral C++ → GA scheduling → VHDL RTL hardware
- ✓ GA reduces waiting time by 7.6% over FIFO scheduling
- ✓ VHDL simulation outputs match C++ reference exactly

Thank you